

Аналіз екосистеми Code-to-Diagram та архітектурних патернів для адаптації під семантику DRAKON

1. Executive Summary

Глибокий архітектурний та технічний аналіз глобальної екосистеми рішень класу "code-to-diagram" виявляє стрімкий розвиток інструментарію, спрямованого на автоматичну візуалізацію програмного коду, структур даних та системної архітектури. Сучасний ринок переживає парадигмальний зсув, спричинений впровадженням стандарту Model Context Protocol (MCP) від Anthropic, який дозволяє великим мовним моделям (LLM) безпосередньо взаємодіяти зі спеціалізованими серверами генерації графів, середовищами розробки та системами контролю версій.¹ Дослідження підтверджує наявність широкого спектра готових компонентів: від утиліт командного рядка (CLI), які виконують глибокий розбір абстрактних синтаксичних дерев (AST)², до високопродуктивних рушіїв компоновання графів (graph layout engines), здатних обробляти тисячі вузлів у браузері.⁴

Незважаючи на багатство існуючих підходів, їх пряма адаптація під візуальну мову програмування DRAKON стикається з фундаментальними топологічними та семантичними перешкодами. Переважна більшість інструментів (таких як Mermaid.js, PlantUML, Graphviz) базується на алгоритмах макетування, які намагаються мінімізувати перетини ліній або збалансувати візуальну масу графа.⁶ Натомість DRAKON вимагає абсолютної відсутності перетинів, жорсткого дотримання ортогональної (Манхеттенської) маршрутизації та концептуального підпорядкування графа центральній вертикальній осі — "шампуру" (happy path).⁶ Більше того, семантика DRAKON передбачає чітку просторову деградацію: альтернативні маршрути та обробники помилок повинні безальтернативно зміщуватися праворуч.⁶ Такі суворі обмеження роблять стандартні текстові DSL (Domain-Specific Languages) та алгоритми гравітаційного макетування абсолютно непридатними для генерації DRAKON-схем без глибокої архітектурної модифікації.

Вирішення цієї проблеми лежить у площині застосування гібридної конвеєрної архітектури (Hybrid Topological Semantic Extraction). Дослідження демонструє, що найуспішнішим підходом є розмежування етапів екстракції логіки та її візуального рендерингу. Детерміновані парсери (наприклад, ts-morph або tree-sitter-graph) повинні перетворювати вихідний код на базові графи потоку управління (CFG).⁸ Великі мовні моделі мають використовуватися виключно для семантичного маркування цих графів — ідентифікації "happy path" та відхилень, формуючи на виході плоский JSON-словник ідентифікаторів (adjacency list).⁶ Це усуває проблему просторових галюцинацій LLM, оскільки модель взагалі не оперує геометричними координатами. Фінальне розміщення

(layout) має делегуватися спеціалізованим алгоритмам на кшталт drakonwidget.js або модифікованому ELK.js (Eclipse Layout Kernel) з налаштуваннями ортогональної маршрутизації та пошарового компонування.⁶ Такий ешелонований підхід дозволяє максимально перевикористати існуючі open-source парсери та рушії візуалізації, зберігаючи при цьому унікальну ергономіку стандартів DRAKON.

2. Solutions Overview

Дослідження ринку та відкритого вихідного коду дозволяє класифікувати існуючі рішення для генерації діаграм з коду на кілька фундаментальних категорій. Кожна категорія застосовує власну методологію перетворення абстрактних даних на візуальні репрезентації, пропонуючи різні рівні контролю, масштабованості та інтегрованості зі штучним інтелектом.

2.1. MCP-сервери та AI-інтегровані платформи

З появою Model Context Protocol (MCP) екосистема інструментів отримала стандартизований шар абстракції, що працює за архітектурою "клієнт-сервер" на базі JSON-RPC.¹ MCP-сервери капсулюють логіку доступу до ресурсів та інструментів генерації, дозволяючи LLM-агентам (наприклад, Claude Code або Cursor) динамічно будувати діаграми під час діалогу з розробником.¹

Одним із найпомітніших рішень у цій категорії є платформа ToDiagram та її спеціалізований MCP-сервер (@todigram/todigram-mcp). Цей інструмент оптимізований для перетворення різноманітних структур даних, таких як JSON, YAML, XML та CSV, на інтерактивні візуальні діаграми.¹¹ Сервер дозволяє інтегруватися безпосередньо в середовище AI-копайлотів та CI/CD конвеєрів для автоматичної генерації архітектурних карт або відображення конфігураційних змін (diffs).¹¹ Хоча ToDiagram демонструє високий рівень автоматизації та має вбудовану функцію "Text to Diagram AI"¹¹, його використання обмежене закритою хмарною інфраструктурою. Рендеринг відбувається на віддалених серверах, а система покладається на власні закриті алгоритми розміщення, що робить неможливим впровадження жорстких топологічних правил DRAKON.¹¹ Водночас, патерн ToDiagram щодо використання MCP для прийому кастомних JSON-payloads і їх валідації через JSON Schema є еталонним прикладом для розробки власного локального сервера.¹¹

Значний інтерес становить Draw.io MCP Server (lgazo/drawio-mcp-server), який відкриває AI-агентам доступ до програмного керування елементами діаграм Draw.io.¹² На відміну від систем, що генерують граф одноразово як статичне зображення, цей сервер підтримує ітеративні мутації. Агенти можуть створювати нові вузли, з'єднувати їх лініями, керувати шарами (наприклад, фоновими елементами) та працювати з кількома сторінками документа одночасно.¹² Особливістю архітектури є використання FIFO (First-In-First-Out) серіалізації для операцій, що гарантує цілісність даних при паралельній роботі кількох агентів або при локальному редагуванні.¹² Цей підхід ідеально демонструє, як можна керувати станом великого полотна, проте він покладається на здібності LLM генерувати

XML-структури та орієнтуватися у просторі, що часто призводить до візуального хаосу без детермінованого рушія компоновання.

Mermaid.js, де-факто стандарт для генерації діаграм з тексту у Markdown, також має власну MCP-інтеграцію (mermaid.ai). Сервер `validate_and_render_mermaid_diagram` обробляє текстовий DSL (Domain-Specific Language) Mermaid, перевіряє синтаксис та повертає готові SVG- або PNG-зображення.¹³ Протокол підтримує передачу даних через локальний стандартний потік вводу/виводу (stdio) або через Server-Sent Events (SSE) для віддаленої обробки.¹ Недоліком цього рішення в контексті DRAKON є фундаментальна топологічна невідповідність рушія `dagre`, який лежить в основі Mermaid. Він не здатен гарантувати відсутність перетинів та утримувати вертикальний шампур алгоритму.⁶

Для архітекторів програмного забезпечення спеціалізованим інструментом є IcePanel MCP, який фокусується на моделюванні за стандартом C4.¹⁴ LLM використовує цей сервер для отримання відповідей на архітектурні запити та для підтримки синхронізації між кодом і документацією. Схожі можливості має сервер AntV Chart MCP (antvis/mcp-server-chart), який підтримує генерацію розгалужених flow-діаграм та інтелект-карт за допомогою бібліотек візуалізації даних.¹⁵

2.2. CLI-інструменти та бібліотеки статичного аналізу (AST-to-Graph)

Друга категорія рішень охоплює інструменти, які спеціалізуються на детермінованій екстракції логіки з вихідного коду шляхом побудови абстрактних синтаксичних дерев (AST) та їх конвертації у графи. Ці системи працюють без використання LLM, спираючись на жорсткі парсери та евристичні алгоритми.

Проект `code2flow` є визначним прикладом генератора графів викликів (call graphs) для динамічних мов програмування, таких як Python, JavaScript, Ruby та PHP.² Архітектура `code2flow` базується на рекурсивному аналізі: система розділяє вихідний код на логічні простори імен — "групи" (файли, модулі, класи) та "вузли" (функції).² Оскільки в динамічних мовах неможливо точно визначити тип змінної чи область видимості до моменту виконання, `code2flow` застосовує складну систему евристичного вирішення (heuristic-based resolution).² Для кожного виклику функції алгоритм спочатку шукає збіг серед змінних у поточній області видимості. Якщо збіг не знайдено, він здійснює розширений пошук за "одиначим збігом" серед усіх інших груп у проекті.² Після формування зв'язків система виконує очищення графа від "сиріт" (orphan trimming), генеруючи фінальний вихідний код у форматі DOT для рендерингу через Graphviz.²

Більш вузькоспеціалізованим інструментом є `pyflowchart`, який трансліює сирцевий код на Python у текстовий DSL формату `flowchart.js`.¹⁸ На відміну від систем, що будують графи викликів між модулями, `pyflowchart` генерує детальний граф потоку управління (Control Flow Graph) всередині конкретної функції або методу.¹⁸ Бібліотека демонструє важливі архітектурні патерни спрощення графа. Наприклад, параметр `simplify` дозволяє

автоматично згортати однерядкові тіла умов або циклів безпосередньо у вузол умови, що радикально зменшує візуальний шум.¹⁸ Інша функція, `conds-align`, забезпечує вирівнювання послідовних операторів `if`, створюючи більш читабельні структури, що концептуально перегукується з вимогами ергономіки DRAGON.¹⁸

На рівні фундаментального парсингу виділяються інструменти `ts-morph` та `tree-sitter-graph`. `ts-morph` надає потужну об'єктно-орієнтовану обгортку навколо TypeScript Compiler API, що дозволяє виконувати глибокий статичний аналіз, відстежувати залежності компонентів та ідентифікувати шаблони використання.³ Ця бібліотека широко використовується в інструментах рефакторингу та лінтингу.²⁰ Бібліотека `tree-sitter-graph`, розроблена на Rust, йде ще далі. Вона визначає власну предметно-орієнтовану мову (DSL), яка дозволяє конструювати довільні структури графів із синтаксичних дерев коду, розібраного за допомогою швидких парсерів `tree-sitter`.⁸ Висока продуктивність, підтримка практично будь-якої мови програмування та здатність гнучко моделювати Control Flow Graphs роблять `tree-sitter-graph` потужним інструментом для побудови фундаменту конвеєра `code-to-diagram`.⁸

2.3. Рушії макетування та рендерингу (Graph Layout Engines)

Візуалізація згенерованих графів у браузері є критичним викликом, який вимагає розрахунку геометричних координат (макетування) та відмальовування елементів на екрані. Ринок поділяється на системи на базі SVG та Canvas, кожна з яких має свої переваги для обробки великих структур.⁴

Найбільш релевантним алгоритмічним ядром для макетування спрямованих графів є Eclipse Layout Kernel (ELK.js). Це потужна бібліотека, зібрана у JavaScript з оригінальної Java-кодової бази, яка підтримує виконання розрахунків у ізольованих Web Workers.⁵ ELK.js пропонує кілька алгоритмів компоновання, серед яких флагманським є `layered` (пошарове макетування на основі методу Сугіяма).⁵ Цей алгоритм враховує напрямок виконання, мінімізує перетини і, що найважливіше, підтримує ортогональну (Манхеттенську) маршрутизацію ліній зв'язку та явні порти кріплення (`ports`) на межах вузлів.⁵ Гнучкість ELK.js дозволяє налаштовувати відступи між вузлами та шарами, роблячи його найбільш зрілим open-source рішенням для імітації жорстких топологічних правил алгоритмів.²²

Для візуального рендерингу та управління взаємодією лідером є екосистема React Flow (та її аналог Svelte Flow).⁴ React Flow забезпечує високопродуктивне відображення вузлів, нескінченне полотно (`infinite canvas`), панорамування, масштабування та віртуалізацію рендерингу (`culling`). Однак важливо зазначити, що React Flow є математично "німим" інструментом: він не вміє самостійно розраховувати координати вузлів.²⁵ У сучасних архітектурах він використовується у тандемі з алгоритмічними рушіями (такими як ELK.js або `dagre`), де рушій обчислює координати `x` та `y`, а React Flow просто переносить їх на екран.²²

На противагу цьому, існують бібліотеки `D3.js` та `Cytoscape.js`, які фокусуються на аналітиці

даних та дослідженнях мереж.⁴ Вони часто використовують гравітаційно-пружинні (force-directed) алгоритми для самоорганізації вузлів. Такий підхід генерує органічні, "плаваючі" структури, які абсолютно суперечать строгій, впорядкованій Манхеттенській геометрії DRAKON, тому ці бібліотеки не можуть бути використані для алгоритмічного програмування.⁴ Enterprise-рішення на кшталт GoJS або JointJS також пропонують потужні можливості візуалізації та Манхеттенську маршрутизацію, але часто мають комерційні ліцензії, що обмежує їх застосування у відкритих продуктах.⁴

3. Solutions Comparison Table

Наведена таблиця містить комплексне архітектурне порівняння існуючих рішень. Оцінка "DRAKON Adaptation Potential" враховує здатність системи відповідати фундаментальним правилам мови (відсутність перетинів, наявність центральної осі, ортогональність ліній та деградація вправо).⁶

Назва	Тип	Input	Output	Open-source	MCP	Локальний режим	DRAKON Adaptation Potential	Примітки
todiagram-mcp	Хмарний MCP-сервіс	JSON, YAML, CSV, Custom Payloads	Interactive Web UI	Ні (закритий рушій) ¹¹	Так ¹¹	Ні (лише через API key)	Дуже низький. Рендеринг повністю інкапсульований у закритій хмарі. Немоżliв	Архітектурно корисний як зразок обробки Custom JSON-схем та інтегр

							о втрут итися в логіку макет уванн я або нав'яз ати Манх еттен ську топол огію "шам пура". ¹¹	ації з AI агент ами.
mer maid. ai (MCP)	Бібліо тека / MCP Serve r	Merm aid DSL (Text)	SVG, PNG, Intera ctive Link	Частк ово (Mer maid відкр итий, AI-хм ара закри та) ¹³	Так ¹³	Так (чепе з studio) ¹³	Низь кий. Merm aid спир аєтьс я на алгор итми dagre , які не забез печу ють 100% відсу тність перет инів ліній і не здатн	Вели кі мовні моде лі часто припу скают ься синта ксичн их поми лок при генер ації склад ного Merm aid DSL.

							і ізолю вати happy path на прямі й осі.	
drawi o-mc p-ser ver	MCP Serve r (UI auto matio n)	Мута ційні кома нди (Natu ral Langu age, XML)	Draw.i o XML	Так ¹²	Так ¹²	Так (вбуд овани й редак тор)	Сере дній. Нада є повни й контр оль над елем ентам и (коор динат и, стилі) , але не має мате матич ної валід ації DRAK ON. AI-аг ент повин ен сам обчи слов	FIFO- серіа лізаці я дозво ляє безпе чне ітера тивне редаг уванн я графа кільк ома агент ами. ¹²

							ати геоме трію.	
code 2flow	CLI / Бібліо тека (екст ракці я)	Pytho n, JavaS cript, Ruby, PHP кодов і бази	DOT- файл и (Grap hviz)	Так (MIT) ²⁷	Ні	Так	Висо кий (як екст ракт ор логік и). Еврис тичні мето ди стати чного аналі зу (AST) та відст ежен ня обла стей види мості є ідеал ьним и для створ ення фунд амен ту графа ² .	Ігнор ує runti ме-з начен ня (напр иклад , фабр ичні функ ції), зали шає обро бку сема нтики нероз крито ю. ²

pyflowchart	CLI / Бібліотека	Python Source Code	flowchart.js DSL	Так ¹⁸	Ні	Так ¹⁸	Середній. Генерує DSL з оптимізаціями на зразок simplify та cond-align, що підвищує читабельність. ¹⁸ Працює тільки з однією мовою.	Фокусується на внутрішньому потоці логіки (CFG), що семантично ближче до DRAGON, ніж call graphs.
tree-sitter-graph	Бібліотека (Парсинг)	Сирцевий код будь-якої популярної мови	Arbitrary Graph (через DSL)	Так (MIT / Apache) ⁸	Ні	Так ⁸	Дуже високий. Дозволяє визначити власні правила розб	Вимагає глибокого програмування конфігурацій DSL для кожн

							ору (DSL), перетворюючи будь-який AST безпосередньо у плоский JSON словник, готовий для LLM. ⁸	ого цільового синтаксису.
ELK.js	Layout Engine (Макетування)	JSON (структура без координат x/y)	JSON (з обчисленими x/y та edges)	Так (EPL-2.0) ⁵	Ні	Так	Високий. Алгоритм layered з підтримкою orthogonal edge routing та керуванням портами є найзр	Потребує глибокого налаштування параметрів інтервалів та примусового вирівнювання

							ілішо ю відкр итою альте рнати вою для імітац ії DRAK ON. ⁵	осей.
ts-morph	AST Wrapper (Парс инг)	TypeS cript / JavaS cript	AST Node Objec ts	Так (MIT) ³	Ні	Так	Дуже висо кий. Забез печує об'єк тно-о рієнт овани й досту п до компі лятор а TypeS cript, що дозво ляє безпе чно виокр емлю вати логіку без викор истан	Обме жени й викл ючно екоси стем ою JavaS cript/ TypeS cript.

							ня регул ярних вираз ів. ³	
--	--	--	--	--	--	--	---	--

4. Architecture Patterns Catalog

Аналіз інтеграції систем штучного інтелекту у процес візуалізації розкриває кілька фундаментальних архітектурних патернів. Спроби використовувати LLM для прямої генерації геометричних або візуальних параметрів (наприклад, малювання SVG координат) призводять до критичного рівня галюцинацій та порушень топології, оскільки трансформерні моделі не мають вбудованого просторового 2D-рендеру.⁶ Експертні системи змушені застосовувати багатошарові (ешелоновані) конвеєри обробки, де відповідальність чітко розмежована між генеративним ШІ та детермінованою математикою.

Патерн архітектури	Кроки обробки (Pipeline)	Де використовується LLM	Де використовується Deterministic код	Адаптованість під DRAGON
Textual DSL Generation (Mermaid / PlantUML Pattern)	Code/Text → LLM → DSL String → Layout Engine → Renderer	LLM зчитує вхідні дані та формує синтаксис текстової мови розмітки (наприклад, graph TD; A-->B). ¹³	Парсинг згенерованого DSL, обчислення координат (наприклад, рушієм dagre) та відмальовування кривих у DOM.	Низька. Текстові DSL не мають конструкцій для примусового збереження центрального "шампура" або маркування помилок праворуч. ⁶
Heuristic	Source Code	Не	Усі етапи:	Середня.

AST Extraction (code2flow Pattern)	→ AST Parser → Heuristic Resolver → Graph Model → Graphviz	використовується. Аналіз повністю алгоритмічний і спирається на жорсткі правила синтаксису. ²	побудова дерев, відстеження видимості змінних, зв'язування викликів та усунення "сиріт". ²	Надійно формує дерево залежностей без галюцинацій, але не здатне виокремити бізнес-семантику (яка гілка є основним сценарієм, а яка — помилкою).
LLM as Graph Mutator (Draw.io MCP Pattern)	Context → LLM → JSON-RPC Tool Calls → State Manager → UI	Прийняття рішень щодо виклику специфічних методів модифікації (наприклад, create_node, connect_edges). ¹²	Управління станом полотна, серіалізація змін у XML, запобігання невалідним операціям. ¹²	Середня. Підходить для інтерактивного редагування в діалозі. Проте агенту надзвичайно складно уникати перетинів ліній без математичного зворотного зв'язку про площину. ¹²
Hybrid Topological Semantic Extraction (Recommended for	Code → AST Parser → Filtered IR → LLM → Flat JSON →	Семантичний мапінг: LLM отримує очищену структуру і визначає	AST парсинг (зменшення шуму), валідація JSON-схеми, розрахунок	Максимальна. Плоский JSON-словник ідентифікаторів

DRAKON)	Layout Engine	логічні полі (happy path призначається на "шампур", винятки — вправо). ⁶	координат та математично точний рендеринг геометрії. ⁶	(adjacency list) нівелює просторові галюцинації LLM ⁶ , а детермінований рушій забезпечує топологічні інваріанти.
---------	---------------	---	---	--

У рекомендованому гібридному патерні роль LLM зводиться виключно до семантичного класифікатора. Модель маркує абстрактні логічні блоки відповідними тегамі (наприклад, action, question, error_handler) та формує ідентифікатори лінійних зв'язків між ними, не маючи жодного уявлення про пікселі, відступи чи кути нахилу ліній.⁶

5. Open-source Components Worth Reusing

Для реалізації гібридного конвеєра генерації діаграм DRAKON з вихідного коду нераціонально імплементувати комплексну математику розміщення графів та синтаксичний розбір мов програмування з нуля. Існують зрілі модулі з відкритим вихідним кодом, які покривають ключові етапи пайплайну.

Бібліотека / Компонент	Основне призначення	Мова / Платформа	Ліцензія	Relevance (Придатність) для DRAKON
ts-morph	Конвертація коду в Abstract Syntax Tree (AST). Дозволяє ітерувати по дереву функцій та екстрагувати виклики і	TypeScript / Node.js	MIT	Критично висока. Ідеальний інструмент для генерації початкового графа (до втручання LLM) при аналізі кодових баз

	змінні. ³			на TypeScript/Ja vaScript. ⁹
tree-sitter & tree-sitter-g raph	Універсальни й надшвидкий AST-парсер із підтримкою створення графів поток управління (CFG) через спеціальну DSL-мову. ⁸	Rust / C	MIT / Apache-2.0	Висока. Дозволяє побудувати єдину архітектуру синтаксичног о розбору для Python, Java, Go, C++ та інших мов. ⁸
drakonwidge t.js	Vanilla JS бібліотека клієнтського рендерингу діаграм за суворими правилами стандарту DRAKON. ⁶	JavaScript (Browser DOM / Canvas)	Unlicense / Public Domain ⁶	Абсолютна. Експертно вирішує математичну проблему Манхеттенсь кого макетування (routing) без перетинів та утримує топологію "шампура". ⁶
ELK.js (Eclipse Layout Kernel)	Алгоритмічн е розміщення вузлів (layouting) та складна ортогональн а маршрутизац ія ліній. ⁵	TypeScript / Java (compiled)	EPL-2.0	Висока (як fallback-мех анізм). Якщо можливості drakonwidge виявляться недостатніми , алгоритм layered з параметром

				elk.direction=DOWN здатен наближено імітувати DRAKON. ⁵
Zod	Типізована валідація вхідних структур даних (JSON), що генеруються великими мовними моделями, перед їх відмальовува нням.	TypeScript	MIT	Критично висока. Запобігає фатальним збоям UI, жорстко перевіряючи, чи існують цільові ідентифікато ри (ID), згенеровані LLM, у словнику вузлів items. ⁶
React Flow (або Svelte Flow)	Управління інтерактивни м станом полотна (панорамува ння, масштабуван ня, виділення вузлів). ⁴	TypeScript / React	MIT ²⁴	Середня. Сам React Flow не вміє обчислювати макетування (делегує це ELK або Dagre ²⁵), але чудово підходить як високорівнев а обгортка для створення інтерактивно го редактора.

6. DRAKON Adaptation Analysis

Адаптація стандартних підходів генерації графів (graph-generation) під семантику DRAKON вимагає вирішення кількох фундаментальних конфліктів між класичною топологією та жорсткою когнітивною ергономікою візуального програмування.

6.1. Де DRAKON принципово відрізняється від класичного Flowchart

Класичні стандарти блочного моделювання (наприклад, ISO 5807 або UML) та інструменти візуалізації (Mermaid, Graphviz, PlantUML) фокусуються виключно на відображенні зв'язків як таких.⁶ Алгоритми їх компоновання намагаються лише *мінімізувати* кількість перетинів ліній (crossing reduction algorithm) і збалансувати просторову масу діаграми. DRAKON, навпаки, діє в рамках жорстких топологічних інваріантів⁶:

1. **Політика нульових перетинів (Zero-Intersection Policy):** Перетини комунікаційних ліній категорично заборонені. Мозок людини підсвідомо сприймає будь-яке схрещення як потенційний логічний вузол, і витрачає когнітивні ресурси на спростування цього факту. Рушій DRAKON не має права мінімізувати перетини — він зобов'язаний розширити площину малювання або декомпонувати алгоритм (наприклад, через структуру "Силует"), щоб уникнути їх повністю.⁶
2. **Правило Шампура (The Skewer Rule):** Алгоритми Graphviz (на базі dagre) розташовують вузли так, щоб збалансувати дерево графа. У DRAKON діє непорушне правило: найбажаніший і найчастіше використовуваний шлях виконання (happy path) має формувати ідеальну, пряму вертикальну вісь ("шампур"). Око інженера не повинно здійснювати горизонтальних пошукових рухів для розуміння головної мети функції.⁶
3. **Асиметрія розгалуження та просторова деградація:** У класичному графічному поданні виходи з умови (наприклад, "Так" і "Ні") є візуально рівнозначними. У DRAKON діє евристика когнітивної деградації: "чим правіше, тим гірше" (The further right, the worse). Це означає, що система повинна усвідомлювати семантику потоку. Блоки обробки фатальних помилок, винятки (catch (e)) та малоімовірні сценарії повинні примусово виштовхуватися на крайню праву периферію діаграми.⁶

6.2. Непридатність стандартних підходів для AI

Спроба використання LLM для прямого генерування SVG-коду або розрахунку точних піксельних X/Y координат для складних алгоритмічних схем приречена на провал. Трансформерні архітектури чудово оперують текстовою семантикою, але не мають внутрішнього двовимірного просторового рушія, що неминуче призводить до візуальних "галюцинацій" (накладання вузлів, переплутані стрілки).⁶

Крім того, моделям штучного інтелекту важко утримувати контекст глибоко вкладених структур (наприклад, сирих абстрактних синтаксичних дерев — AST). Згенерована дужка,

що закривається не в тому місці, руйнує весь логічний ланцюг.⁶

6.3. DRAKON-специфічна адаптація в конвеєрі

Для забезпечення стабільної генерації DRAKON-діаграм необхідно архітектурно адаптувати процес обміну даними:

- **Відмова від AST як формату обміну:** Абстрактне синтаксичне дерево є занадто деталізованим і містить багато синтаксичного шуму. Парсери (наприклад, code2flow або ts-morph) повинні попередньо згортати (simplify) лінійні операції (декларації, присвоєння) в єдині об'єднані вузли перед передачею в контекст LLM. Цей патерн успішно застосовується в параметрі simplify=True бібліотеки pyflowchart.¹⁸
- **Використання плоских словників (Adjacency List):** Комунікація між LLM та рушієм рендерингу має відбуватися виключно у форматі плоского JSON-об'єкта (Map/Dictionary). У такій структурі ключами є унікальні строкові ідентифікатори вузлів, а значеннями — об'єкти з метаданими (текст) та посиланнями one (наступний крок по шампуру) і two (альтернативний крок вправо).⁶ Це мінімізує проблеми вкладеності.
- **Семантичний Prompt Engineering:** Завдання LLM кардинально змінюється. Вона отримує AST або текст, і має лише класифікувати логічні ролі вузлів. Згідно з правилом "чим правіше, тим гірше", модель повинна навчитися ідентифікувати "успішний сценарій" і спрямовувати його через топологічний атрибут one, а всі відхилення — через атрибут two.⁶
- **Математичний Рендеринг:** Валідований плоский JSON передається до детермінованого рушія (наприклад, drakonwidget.js), який здійснює обчислення метрик, розтягує блоки, будує шампури та гарантує ідеальне Манхеттенське трасування ліній без перетинів, повністю ізолюючи AI від геометрії.⁶

7. Proposed Reference Architecture

Архітектура перспективного генератора Code-to-DRAKON повинна бути спроектована як багатоетапний конвеєр обробки даних (Data Pipeline), де кожен інструмент має чітко обмежену відповідальність, а переходи між етапами суворо контролюються валідаторами.

Псевдосхема конвеєра генерації (Pipeline Architecture):

Етап (Processing Step)	Вхідні дані (Input)	Компонент / Інструмент	Вихідні дані (Output)	Відповідальність та Логіка
1. Source	Вихідний код	ts-morph або	Abstract	Детермінова

Analysis	(наприклад, JS, Python)	tree-sitter ³	Syntax Tree (AST) або Control Flow Graph (CFG)	ний лексичний розбір вихідного коду. Ідентифікація функцій, змінних, умовних переходів та циклів.
2. Simplification	Деталізований AST	Кастомний скрипт (Heuristics на зразок code2flow) ²	Спрощений текстовий граф (Згорнуті блоки)	Злиття дрібних імперативних операцій (наприклад, послідовних математичних присвоєнь) в єдині об'єднані вузли. Це радикально зменшує візуальний шум (патерн simplify) ¹⁸ та економить токени контекстного вікна LLM.
3. Semantic Mapping	Спрощений граф + System Prompt	LLM (напр. інтеграція через MCP ¹)	Згенерований JSON (Adjacency List)	LLM виступає як аналітик бізнес-логіки. Вона виокремлює "Happy path", призначає ідентифікатори

				топологічних вказівників (one для продовження , two для помилок), та генерує читабельний текст для контенту іконок. ⁶
4. Validation	Згенерований JSON	Zod (TypeScript)	Строго валідований JSON Словник (items)	Перевірка цілісності зв'язків: схема гарантує, що поля one та two вказують виключно на існуючі ідентифікатори у словнику, а також перевіряє наявність обов'язкових термінальних вузлів (Start/End). ⁶
5. Layout & Geometry	Валідований JSON Словник	drakonwidget.js ⁶ або адаптований ELK.js ⁵	Топологічний граф з розрахованими X/Y координатами та routing-шляхами	Динамічний математичний розрахунок метрик. Позиціонування елементів на осі "шампура", прокладання

				ортогональних ліній, розширення площини для уникнення будь-яких перетинів. ⁶
6. UI Rendering	Об'єкти з координатами та метаданими	React Flow ²⁴ (як Event Host) або нативний DOM/Canvas	Інтерактивний, редагований граф у браузері користувача	Рендеринг пікселів на екран. Забезпечення інтерактивності: можливість користувачу вручну редагувати текстовий контент іконок, переміщувати вузли, панорамувати та масштабувати полотно.

Ця ешелонована (розшарована) архітектура гарантує стабільність системи. Оскільки генеративна модель (LLM) взагалі не має доступу до візуального шару та розрахунку фізичних відстаней між елементами, ризик створення топологічно невалідних структур (галюцинацій макету) зводиться до нуля.

8. What NOT to Reuse and Why

Під час розробки специфікації (Engineering Spec) критично важливо уникнути спокуси використати популярні, але архітектурно несумісні інструменти. Використання наступних компонентів та патернів призведе до руйнування унікальної семантики мови DRAKON.

- **Серверна спадщина DRAKON (DrakonHub Backend):** Бекенд хмарного редактора DrakonHub базується на платформі in-memory баз даних Tarantool. Бізнес-логіка

цього бекенду написана мовою Lua, код якої, у свою чергу, автоматично згенерований з графічних моделей редактора на Tcl/Tk.⁶ Ця застаріла self-hosted архітектура є "абсолютно непідтримуваною" і виступає відвертим антипатерном для сучасних мікросервісних SPA-проектів. Для інтеграції придатний лише ізольований клієнтський віджет `drakonwidget.js`.⁶

- **Mermaid.js та PlantUML:** Попри те, що це де-факто стандарти індустрії для парадигми "Diagram-as-Code" ⁴, їхні внутрішні рушії (часто на базі бібліотеки `dagre`) намагаються оптимізувати простір, створюючи "павутиння" зв'язків. Вони допускають перетини ліній при складному розгалуженні та не здатні жорстко ізолювати основний потік на єдиній прямій осі, що є прямим порушенням правила "Шампура".⁶
- **D3.js, Cytoscape.js та Force-Directed Layouts:** Бібліотеки, що використовують фізичні (гравітаційно-пружинні) моделі макетування, де вузли притягуються або відштовхуються як магніти, генерують органічні, хаотичні форми.⁴ Такий підхід абсолютно суперечить базовій вимозі DRAGON щодо використання виключно ортогональної (Манхеттенської) геометрії, де всі кути повинні дорівнювати 90 градусам.⁶
- **Закриті хмарні MCP-сервери (todiagram, Miro):** Такі платформи здатні генерувати якісні блок-схеми та пропонують готові AI-інтеграції через протокол MCP.¹¹ Однак їхня замкнута архітектура рендерингу (vendor lock-in) унеможлиблює втручання в алгоритми макетування для нав'язування правил DRAGON. Крім того, передача корпоративного вихідного коду на зовнішні сервіси часто порушує політики конфіденційності.
- **Пряма генерація графа силами LLM з "сирого" коду:** Відправлення LLM запиту у стилі "ось файл на 2000 рядків, побудуй логічний граф DRAGON" є хибною стратегією. Трансформерні моделі мають обмежене вікно уваги; вони гублять дрібні логічні умови та починають генерувати синтаксичні галюцинації при обробці великих масивів коду. LLM має працювати лише з попередньо очищеними, детерміновано побудованими деревами (AST), сформованими парсерами на кшталт `ts-morph`.⁶

9. Open Questions (Unresolved Questions)

Незважаючи на глибокий теоретичний та архітектурний аналіз існуючих рішень, перед переходом до етапу специфікації інженерної архітектури залишаються невирішеними кілька складних технічних питань, що потребуватимуть розробки прототипів (Proof of Concept):

1. **Ліміти продуктивності та масштабованості макетування:** Наскільки ефективно працює детермінований алгоритм Манхеттенської маршрутизації ліній у `drakonwidget.js` (або `ELK.js`) на екстремально великих графах (понад 1000-2000 вузлів)? Чи спричинить гібридний рендеринг перевантаження DOM-дерева браузера, і чи потребуватиме архітектура впровадження механізмів віртуалізації полотна (virtual viewport culling / React Flow) для стабільної роботи?

2. **Обробка багатозначної типізації (Ambiguity Resolution) в динамічних мовах:** Як саме конвеєр має реагувати на мови з гнучкою типізацією (JavaScript, Python), де AST-парсер не може точно визначити тип або походження об'єкта до моменту виконання (runtime)? Наскільки глибоко необхідно імплементувати евристичні алгоритми пошуку залежностей (подібні до методу "single match" у code2flow²) ще до того, як скорочений контекст буде передано в LLM?
3. **Зворотне відтворення стану (Bi-directional Code Sync):** Якщо інтерактивний UI редактора дозволить користувачеві фізично змінити структуру згенерованого графа (наприклад, перетягнути блок обробки помилки в іншу гілку умовного оператора), як математично правильно транслювати ці топологічні мутації назад у зміни сирцевого вихідного коду? Чи достатньо стандарту JSON-RPC (через MCP) для безпечного автоматичного рефакторингу кодової бази агентом?
4. **Кодування макроікон у плоскому представленні (IR):** Чи здатен запропонований формат плоского JSON-словника ідентифікаторів (items adjacency list) коректно підтримувати складні ієрархічні макроструктури мови DRAKON (наприклад, просторові "Силуети" або циклічні макроікони "For" з валентними точками)? Як саме генеративна модель має позначати межі таких структур (точку входу та виходу з циклу) без використання прямої структурної вкладеності JSON-об'єктів?

10. Curated Bibliography

Для подальшої практичної розробки та вивчення технічних імплементаций рекомендується спиратися на наступний курований перелік ключових інструментів та репозиторіїв з відкритим кодом, проаналізованих у цьому дослідженні:

- **code2flow** (scottrogowski/code2flow): Еталонний генератор графів викликів для динамічних мов (Python, JS, Ruby, PHP). Важливий для вивчення евристичних методів вирішення просторів імен та обробки AST.²
- **pyflowchart** (marshmallow-code/pyflowchart): Вузькоспеціалізована бібліотека для трансляції логіки Python-функцій у формат flowchart.js. Демонструє критичні патерни оптимізації графів: simplify (згортання однорядкових тіл умов) та conds-align (вирівнювання операторів).¹⁸
- **ELK.js / Eclipse Layout Kernel** (kieler/elkjs): Високопродуктивний рушій компоновання графів у JavaScript. Алгоритм layered з підтримкою портів (ports) та ортогональної маршрутизації ліній (orthogonal edge routing) є найкращою відкритою альтернативою для побудови архітектури DRAKON на бекенді.⁵
- **tree-sitter-graph** (tree-sitter/tree-sitter-graph): Універсальна бібліотека на Rust для конструювання довільних графів потоку управління (CFG) безпосередньо з розібраних синтаксичних дерев tree-sitter з використанням власного DSL. Найшвидший спосіб розбору багатомовних кодових баз.⁸
- **ts-morph** (dscherret/ts-morph): Потужна об'єктно-орієнтована обгортка для TypeScript Compiler API. Ідеальний інструмент для детермінованого розбору

фронтенд-коду та побудови початкового AST-дерева до залучення LLM.³

- **Draw.io MCP Server** (lgazo/drawio-mcp-server): Зразок високоякісної імплементації Model Context Protocol. Демонструє, як AI-агент може ітеративно взаємодіяти з візуальним графом за допомогою мутацій стану, керування шарами та багатосторінковим простором документа.¹²

Фінальне Резюме (Executive Summary)

Аналіз сучасного ринку інструментів code-to-diagram доводить, що прямолінійне використання популярних рушіїв (на кшталт Mermaid.js або PlantUML) є тупиковою стратегією для генерації DRAGON-схем, оскільки їхні алгоритми гравітаційного компонування не здатні підтримувати жорсткі топологічні інваріанти (абсолютну заборону на перетини ліній та утримання "шампура" на ідеальній вертикальній осі). Водночас спроби навчити великі мовні моделі (LLM) генерувати просторові SVG-координати призводять до критичних галюцинацій геометрії.

Найперспективнішою і єдино життєздатною архітектурою є **гібридний конвеєр семантичної екстракції** (Hybrid Topological Semantic Extraction). У цій парадигмі детерміновані парсери (ts-morph, tree-sitter-graph) виокремлюють з вихідного коду спрощені дерева логіки. Великі мовні моделі застосовуються виключно для семантичного маркування (ідентифікації "happy path" та винятків), формуючи на виході абстрактний плоский JSON-словник. Відповідальність за фінальний математичний розрахунок координат та ортогональну Манхеттенську маршрутизацію ліній делегується спеціалізованим алгоритмам розміщення (drakonwidget.js або налаштованому ELK.js), що гарантує бездоганне дотримання ергономічних правил DRAGON без ризику спотворення візуального подання штучним інтелектом.

Structured Findings

- **Ринок перенасичений, але топологічно несумісний:** Існує величезна кількість інструментів для відображення інфраструктури з коду (Mermaid.js, Graphviz), проте їхні алгоритми компонування допускають перетини ліній при складному розгалуженні та не підтримують строгу Манхеттенську геометрію DRAGON.⁶
- **Небезпека прямих LLM-макетів:** Трансформерні моделі штучного інтелекту позбавлені вбудованих механізмів просторового розрахунку. Генерація X/Y координат або складного графа силами самої LLM неминуче призводить до візуальних "галюцинацій" (накладання вузлів).⁶
- **Плоский JSON як рятівний стандарт (IR):** Обмін даними між парсером коду, LLM та рушієм рендерингу має відбуватися у форматі плоского словника ідентифікаторів (adjacency list). Це радикально знижує ризик втрати структури з боку LLM у порівнянні з глибоко вкладеними деревами (AST).⁶

- **Обов'язкова попередня компресія AST:** LLM має отримувати не сирий вихідний код, а вже очищене абстрактне синтаксичне дерево (AST). Парсери повинні застосовувати алгоритми згортання дрібних операцій (патерн `simplify=True` з `pyflowchart`¹⁸) для економії токенів і уникнення синтаксичного шуму.
- **Детерміновані рушії рендерингу:** Бібліотека `drakonwidget.js` залишається найкращим спеціалізованим рішенням для браузерного рендерингу топології DRAKON.⁶ Як потужну відкриту альтернативу для макетування на стороні сервера варто розглядати модуль `ELK.js` з його пошаровим (`layered`) алгоритмом.⁵
- **Відмова від серверної спадщини:** Оригінальні десктопні кодогенератори Міткіна (на `Tcl/Tk`) та бекенди (на `Tarantool/Lua`) є архаїчними та не підлягають сучасній підтримці. Система генерації має розбудовуватися як новий незалежний мікросервісний конвеєр.⁶

Unresolved Questions

1. **Ліміти продуктивності браузерного рендерингу:** Наскільки стабільно працює Манхеттенська маршрутизація ліній (`drakonwidget.js` / `ELK.js`) на гіпер-масштабних алгоритмічних масивах (понад 1500 вузлів), і чи вимагатиме UI-шар впровадження жорсткої віртуалізації полотна (`viewport culling`) для збереження частоти оновлення кадрів?
2. **Обробка динамічної типізації:** Як парсер (`AST extractor`) оброблятиме поліморфні об'єкти та динамічні типи у мовах на кшталт Python чи JavaScript без середовища виконання? Наскільки складними мають бути евристичні "single match" (за патерном `code2flow`²) до моменту передачі графа в LLM?
3. **Зворотна трансляція мутацій:** Якщо користувач вносить ручні зміни в інтерактивний граф DRAKON (наприклад, переміщує блок обробки винятку в іншу гілку), як розробити безпечний та детермінований механізм синхронізації цих топологічних змін назад у сирцевий вихідний код за допомогою LLM-агентів?

Recommended Next Research Tasks

Завдання, які необхідно врахувати під час розробки технічного завдання (Engineering Spec) у Промпті 4:

1. **Прототипування AST-компресора:** Розробити базовий Proof of Concept (PoC) скрипт на базі бібліотеки `ts-morph` або `tree-sitter-graph`, який зчитує тестовий файл, виконає злиття послідовних лінійних команд у єдині макроблоки та сформує компактний JSON-каркас графа для оптимізації роботи LLM.
2. **Розробка Zod Validation Schema:** Написати сувору типізовану схему (TypeScript `Zod`), яка слугуватиме бар'єром безпеки між LLM та фронтом. Схема повинна жорстко перевіряти цілісність топологічних вказівників (щоб поля `one` та `two`

вказували лише на існуючі ключі в об'єкті items) та валідувати дозволений перелік типів іконок.⁶

3. **Інжиніринг системного промпта (System Prompt) для DRAKON:** Створити еталонний набір інструкцій для LLM. Промпт має навчити модель розпізнавати "happy path" (завжди спрямовуючи його через атрибут one) та застосовувати правило "чим правіше, тим гірше", відкидаючи всі винятки і обробники помилок у бічні гілки (через атрибут two).⁶
4. **Лабораторне тестування ELK.js:** Провести технічний експеримент з конфігурацією ядра Eclipse Layout Kernel. Мета — перевірити, чи здатен алгоритм layered з параметром напрямку DOWN та увімкненою orthogonal edge routing гарантовано створювати безперетинні графи, що візуально імітують вимоги стандарту DRAKON.⁵

Джерела

1. MCP: Model Context Protocol - SFEIR Institute, доступ отримано травня 9, 2026, <https://institute.sfeir.com/en/claude-code/claude-code-mcp-model-context-protocol/>
2. scottrogowski/code2flow: Pretty good call graphs for ... - GitHub, доступ отримано травня 9, 2026, <https://github.com/scottrogowski/code2flow>
3. SiroSuzume's TypeScript Refactoring MCP Server: An AI Engineer's Deep Dive - Skywork, доступ отримано травня 9, 2026, <https://skywork.ai/skypage/en/typescript-refactoring-ai-engineer/1980437487910768640>
4. The Best JavaScript Diagramming Libraries for 2026: From Mermaid to React Flow, доступ отримано травня 9, 2026, <https://awplife.com/top-10-javascript-diagramming-libraries/>
5. kieler/elkjs: ELK's layout algorithms for JavaScript · GitHub - GitHub, доступ отримано травня 9, 2026, <https://github.com/kieler/elkjs>
6. Дослідження мови візуального програмування DRAKON.pdf
7. PlantUML vs Mermaid? : r/ExperiencedDevs - Reddit, доступ отримано травня 9, 2026, https://www.reddit.com/r/ExperiencedDevs/comments/1k7ki6k/plantuml_vs_mermaid/
8. GitHub - tree-sitter/tree-sitter-graph: Construct graphs from parsed ..., доступ отримано травня 9, 2026, <https://github.com/tree-sitter/tree-sitter-graph>
9. codemods-at-scale.md - GitHub, доступ отримано травня 9, 2026, <https://github.com/stevekinney/stevekinney.net/blob/main/courses/enterprise-ui/codemods-at-scale.md>
10. Layout Options (ELK) - The Eclipse Foundation, доступ отримано травня 9, 2026, <https://eclipse.dev/elk/reference/options.html>
11. ToDiagram MCP Server, доступ отримано травня 9, 2026, <https://todigram.com/mcp>
12. Draw.io - Awesome MCP Servers, доступ отримано травня 9, 2026, <https://mcpservers.org/servers/lgazo/drawio-mcp-server>

13. Mermaid Chart | MCP Server, доступ отримано травня 9, 2026,
<https://mermaid.ai/docs/ai/mcp-server>
14. Top MCP tools for software architects | by IcePanel - Medium, доступ отримано травня 9, 2026,
<https://icepanel.medium.com/top-mcp-tools-for-software-architects-20a69f220a5d>
15. Chart - Awesome MCP Servers, доступ отримано травня 9, 2026,
<https://mcpservers.org/servers/antvis/mcp-server-chart>
16. code2flow Alternatives - Python Code Analysis | LibHunt, доступ отримано травня 9, 2026, <https://python.libhunt.com/code2flow-alternatives>
17. Automatic flowchart tool [closed] - Stack Overflow, доступ отримано травня 9, 2026, <https://stackoverflow.com/questions/22411136/automatic-flowchart-tool>
18. pyflowchart - PyPI, доступ отримано травня 9, 2026,
<https://pypi.org/project/pyflowchart/>
19. Awesome Angular - Gitee, доступ отримано травня 9, 2026,
<https://gitee.com/awesome-lib/awesome-angular>
20. makotot/canopy-annotator-client-boundary 0.3.2 on npm - Libraries.io, доступ отримано травня 9, 2026,
<https://libraries.io/npm/@makotot%2Fcanopy-annotator-client-boundary>
21. Enhancing the Robustness of LLM-Generated Code: Empirical Study and Framework - arXiv, доступ отримано травня 9, 2026,
<https://arxiv.org/html/2503.20197v1>
22. Angular ELK.js Graph Layout Example - Foblex Flow, доступ отримано травня 9, 2026, <https://flow.foblex.com/examples/elkjs-layout>
23. xyflow/awesome-node-based-uis - GitHub, доступ отримано травня 9, 2026,
<https://github.com/xyflow/awesome-node-based-uis>
24. React Flow: Node-Based UIs in React, доступ отримано травня 9, 2026,
<https://reactflow.dev/>
25. Elkjs Tree - React Flow, доступ отримано травня 9, 2026,
<https://reactflow.dev/examples/layout/elkjs>
26. Recently Active 'jointjs' Questions - Stack Overflow, доступ отримано травня 9, 2026, <https://stackoverflow.com/questions/tagged/jointjs?tab=Active>
27. 16st58/code_flowchart: Simple tool for creating flowcharts of code - GitHub, доступ отримано травня 9, 2026, <https://github.com/16st58/code-flowchart>
28. ctix (2.7.1) - npm Package Quality | Cloudsmith Navigator, доступ отримано травня 9, 2026, <https://cloudsmith.com/navigator/npm/ctix>
29. MCP Registry | Miro - GitHub, доступ отримано травня 9, 2026,
<https://github.com/mcp/miroapp/mcp-server>